AI ENGINEER · DATA · WEEK 4, DAY 7

Agents in Azure AI Foundry

Agents vs pipelines, plan then act, memory and skills, in Azure AI Foundry

4 hours · instructor-led · hands-on lab

BEFORE WE START

Hand in assignment A2

Answer key

10 questions, each with reviewed SQL and its number

Results table

SqlAgent right; MetricsAgent answered, right, refused

One new number

Formula, tables, rules, date column, grain in one sentence

One-page note

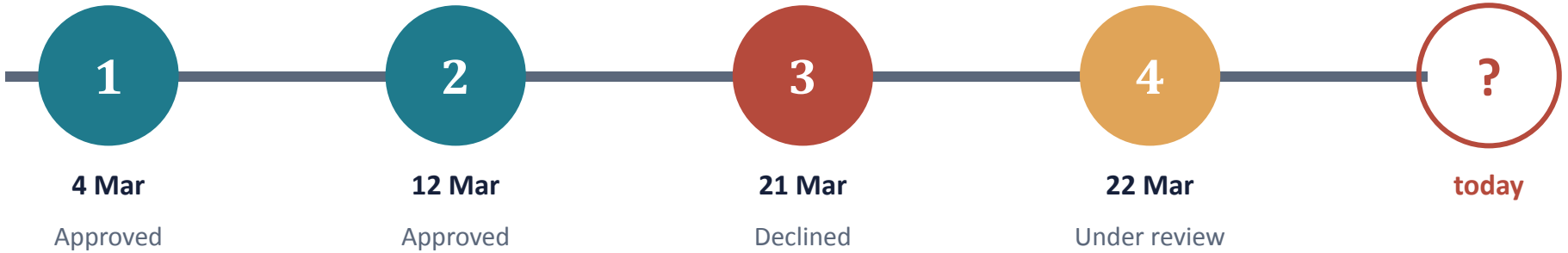
What you found, honestly

In chat: one question MetricsAgent refused, and the number you defined for it.

A2 extended Lab 6. Today the agent stops only answering questions and starts deciding cases.

Running example: should customer 57's fifth claim be paid?

Customer 57, Ishaan S, Kolkata: March 2024 claims. In Lab 7 this is case 1



Refund policy, section 3. "A customer may make up to three refund claims in any rolling thirty day period. A fourth and any subsequent claim within that period is automatically declined ... and must be escalated to a support supervisor."

ASK THE ROOM

The biryani arrived spilled again. What should support tell customer 57, and what did you need to know?

TODAY

Today's topics: pipelines, agents, planners, memory and skills

A. Pipelines

Two agents in a fixed order: policy, then email

B. Agents

One agent chooses its own steps: customer, policy, decision

C. Planner-executor

The same agent writes a plan, does it, checks it

D. Memory and context

The agent remembers across chats, for better and worse

The same cases across the designs, one right answer each. Plus a look at skills and at agents calling agents, ready for Day 8.

We score designs on the same cases, not on how clever they sound.

1

Agents vs pipelines

Fixed steps, or steps the model chooses

Agents vs pipelines: the problem and two designs

The problem: a support worker asks whether a refund should be paid

"I've had 3 refunds this month. Can I claim another one?"

needs: the policy

"Customer 57 ... the biryani arrived spilled. Can they get a refund?"

needs: the policy + the customer's claims + the order date

Pipeline: the developer fixes the steps

The same steps in the same order for every request, decided when it is built.

Good at: Predictable, cheap, easy to test

Weak at: Blind to anything no step was built to look up

Agent: the model chooses the steps

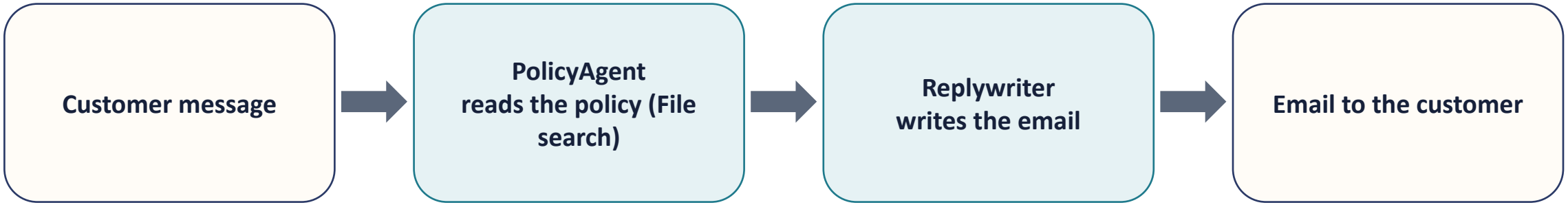
A loop: decide the next step, call a tool, read the result, repeat until it can answer.

Good at: Adapts to the case in front of it

Weak at: Less predictable; more model calls per answer

We build both and ask each the same questions 3 times: a model can answer the same question differently each time.

Pipeline: a fixed sequence of model calls



Built as a Foundry workflow: the first agent's answer is saved as Local.PolicyAnswer and passed to the second.

Good at	Blind to
The same question, asked a thousand times	Anything no step was built to look up

A pipeline is a recipe: predictable, cheap, and only as good as the steps you wrote down.

Pipeline test 1: a question the policy can answer

THE QUESTION *"I've had 3 refunds this month. Can I claim another one?"*

PolicyAgent
searches the refund policy
finds section 3

→

Replywriter
turns that answer
into a polite email

→

Right
in all 3 tries

The right answer

No: a fourth claim within 30 days is declined and goes to a supervisor (policy section 3).

Why the pipeline gets it right

Everything needed is in the question and the policy: the customer told us the count, 3.

This is the question the pipeline was built for.

Pipeline test 2: a question that needs the customer's data

THE QUESTION *"Customer 57 is asking today, 22 March 2024, about order 6987: the biryani arrived spilled. Can they get a refund?"*

What a correct answer needs, and whether the pipeline has it

The rule

Policy section 3: at most 3 claims in 30 days

in File search: the pipeline has it

The count

Customer 57: 4 claims in the last 30 days

in the warehouse: no step looks

The order date

Order 6987 was placed on 21 March

in the warehouse: no step looks

Right answer

Escalate to a supervisor: claim 5 is over the limit.

What the pipeline said

"You are eligible": wrong in all 3 tries. The email is on the next slide.

Why

The customer never said the count, and no step was built to fetch it.

Same pipeline, a different case. Part B gives the agent a tool to look the customer up.

Pipeline on customer 57: the wrong answer

WRONG

Dear customer,

You are eligible to request a refund for the biryani that arrived spilled ... Since your order was delivered on 22 March 2024, your claim is within the acceptable time frame. Please submit photographic evidence within 24 hours of delivery to proceed with your refund.

FreshCart Support

3 of 3 tries

right on "I've had 3 refunds, can I claim another?"

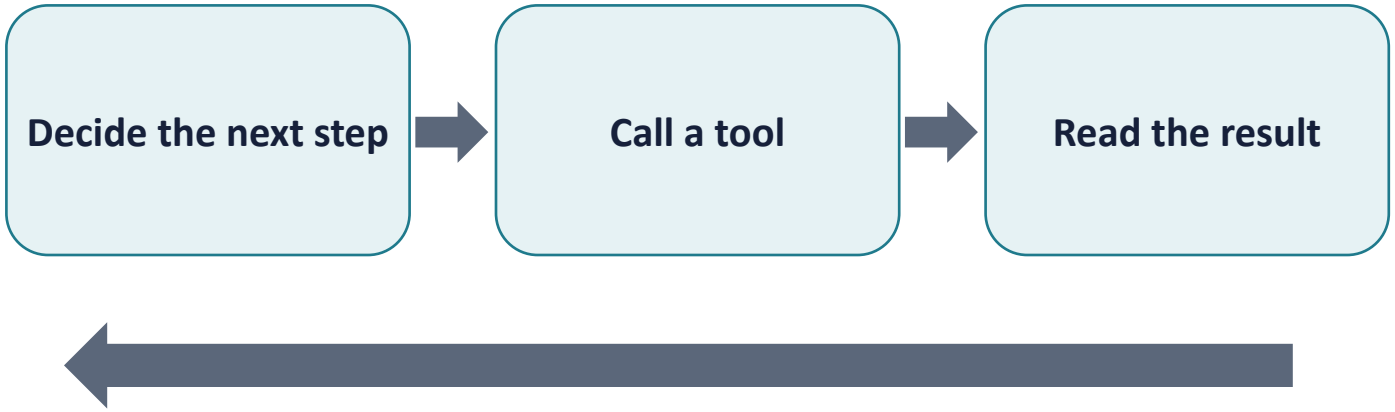
0 of 3 tries

right on customer 57: it said "eligible" every time

Neither step can see the 4 claims. The email even invents a delivery date: the order was delivered on 21 March.

Same pipeline, same tools: perfect on the question it was built for, wrong on the case it was not.

Agent: a loop of reason, act, observe



Its tools in Lab 7

File search

the refund policy, with section numbers

get_customer_claims

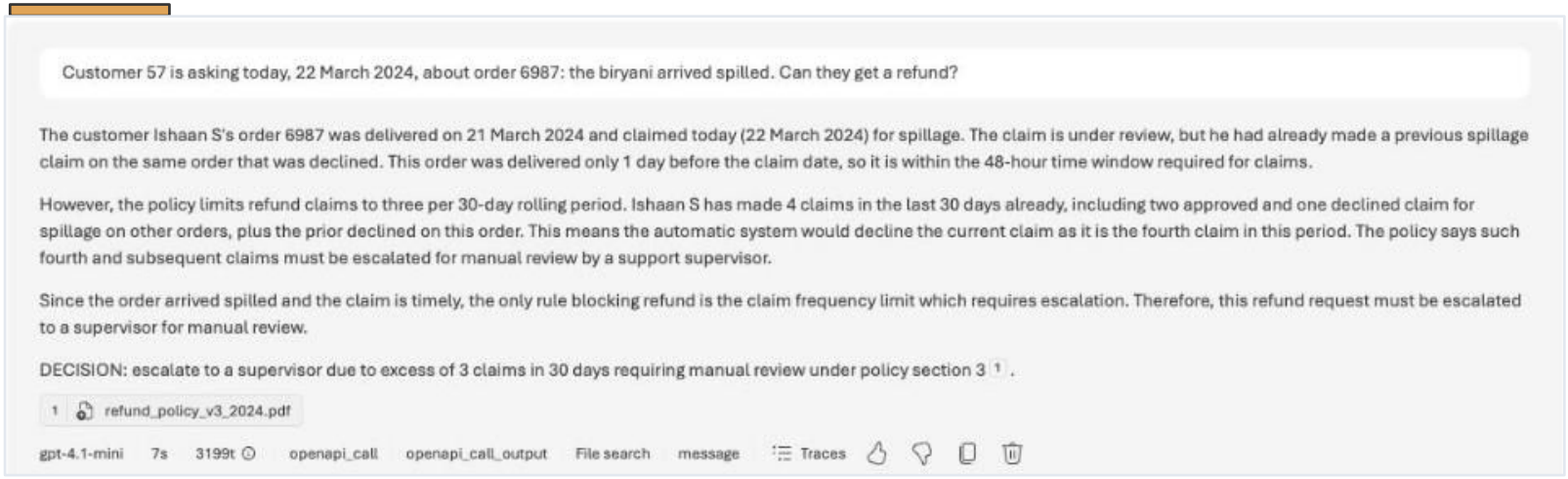
one customer on one day: claims in 30 days, test account, the order

get_official_number

revenue, orders, refund claims, refund rate

The case decides the steps. The tools decide what it can know.

Agent on customer 57: tools first



Escalate, in all 3 tries

The same case the pipeline got wrong every time

4 claims in 30 days

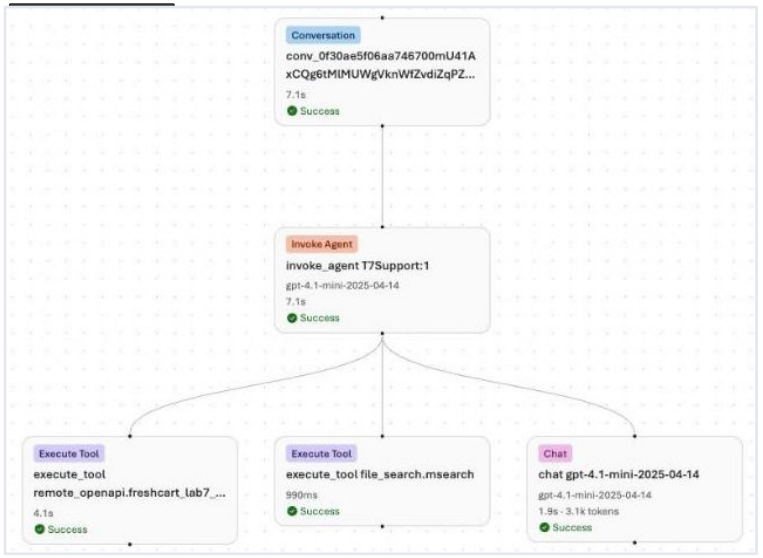
From get_customer_claims, not from the customer

Section 3

Quoted from the policy through File search

Look at the chips under the answer: the customer call, its output, File search, then the message.

Agent traces: the evidence for each step





4.1 s

customer lookup (the database woke up)

1.0 s

policy search

1.9 s

the model writes the answer, 3.1k tokens

Under every answer, click Traces: which tools ran, in what order, with what inputs, and how long.

Pipeline vs agent: scores on the same cases

Case	Pipeline	Agent
1. Customer 57, 4 claims	0 of 3	3 of 3
2. No date given		0 of 10
3. Past 48 hours		2 of 3
4. Two orders, one claim count		0 of 5
5. "My first refund" (data: 3)		3 of 3
7. "I've had three refunds" (data: 0)		3 of 3
8. Test account		3 of 3
9. Refund rate, March		3 of 3
10. Someone else's order		3 of 3

Each cell: right answers in 3 tries (case 2: 10 tries, case 4: 5). Case 3 returns in Part C.

Better is not perfect: the tool was right and the sentence after it was not.

Pipeline vs agent: when to choose each

Pipeline when

- Every case needs the same steps
- You must predict cost and time
- An auditor wants the same path every time

FreshCart: the weekly refund report

Agent when

- The steps depend on the case
- The facts live in several tools
- A wrong step is caught by code, not hope

FreshCart: deciding one customer's refund

Most real systems mix both: a fixed pipeline around an agent that decides one step.

QUICK CHECK

Three questions before planning

1. The pipeline wrote a confident, well-written email that was wrong. Which single change would have fixed case 1?

2. In case 3 the tool returned the right answer and the agent still approved. Where would you look to prove it?

3. Name one FreshCart job that should stay a pipeline, and say why.

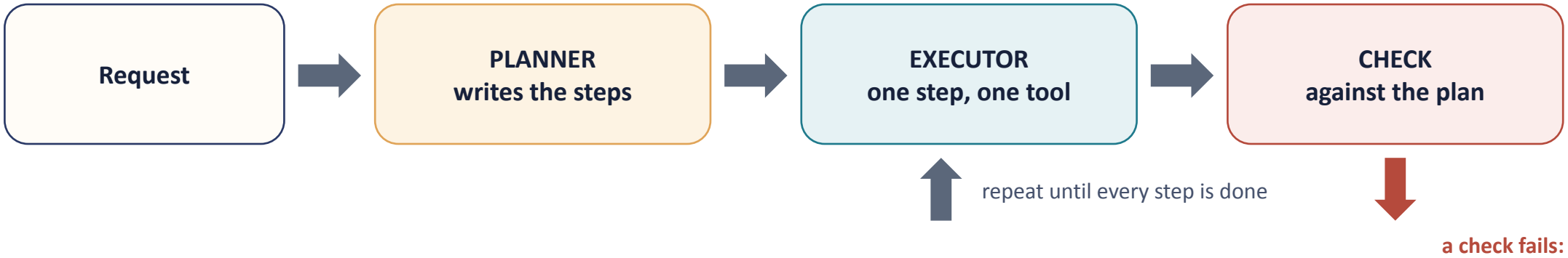
Take answers in chat. Answers in the speaker notes.

2

Planner-executor architectures

Make the agent show its work

Planner-executor architecture: how it works



One agent or two

Ours does all three in one prompt. Larger systems use a planner agent and an executor agent, each with its own model, tools and limits.

When it pays

Long or many-step tasks, steps that depend on earlier results, and work a person must be able to check afterwards.

When it does not

Short lookups. In our tests the planner got the same 11 of 12 answers right as the plain agent, and used about 12% more tokens.

A plan makes the reasoning visible and checkable. It does not make the model smarter.

Pipeline vs agent vs planner-executor: one everyday job

"Book Priya a two-day sales trip to Mumbai next week: flights and a hotel near the client's office, under 40,000 rupees in all."

Pipeline

Steps fixed in advance

1. Search flights for Monday and Tuesday
2. Book the cheapest
3. Book the hotel nearest the office
4. Email the itinerary

Fine on a normal week. When Monday is a holiday or the hotel is full, no step was built for it.

Agent

Decides each next step as it goes

Finds a flight and books it. Then finds the nearby hotel is full, looks further out, and the taxi costs push the total over budget.

It adapts, but it spent money before it knew the total.

Planner-executor

Writes the whole plan first

PLAN: check Priya's calendar, flight options, hotels, taxis, total against 40,000. Shows: 6:05 flight + Hotel Sea View = 36,500. Approved, then DO the bookings, then CHECK each one.

A person sees the whole trip and the total before anything is booked.

Pipeline: the same steps every time. Agent: decides as it goes. Planner: shows the whole plan before it acts.

Pipeline vs agent vs planner-executor: key differences

	Pipeline	Agent	Planner-executor
Who decides the steps	The developer, in advance	The model, one step at a time	The model, all steps first, then it follows them
A case nobody planned for	Fails: it only has the steps it was built with	Handles it, deciding as it goes	Handles it, and shows how before acting
What you can check	The fixed steps, before it runs	The trace, after it ran	The plan before it runs, the trace after
Can a person approve first?	Not needed: the steps never change	No: it acts as it goes	Yes: approve or fix the plan
Cost and speed	Lowest and most predictable	More model calls	Most tokens: a plan, then the calls
Best for	Same steps every time: a weekly report	Short lookups and decisions: one support question	Long jobs or actions that need sign-off: a trip, a refund batch

Pick the simplest design that handles your cases; add a plan when a person must see the steps first.

Planner-executor in one prompt: plan, do, check

PLAN

List the steps and the tool for each. Mark "no tool for this" where none fits.

DO

Run the steps in order. After each tool call, one line on what it returned.

CHECK

Check every fact against a tool result and every rule against a quoted section. Remove what cannot be checked.

Same tools, same model. Only the instructions change: lab7_instructions_4_planner.txt. It also says the agent cannot pay a refund: a person must.

A planner makes the agent's reasoning visible, so a person can check it.

Planner-executor on case 3: the check step

PLAN:

1. Get customer 18's refund claims and order info for order 3405 as of 14 June 2024 from freshcart_lab7_get_customer_claims.
2. Check the refund policy conditions for box crushed/damaged food refund eligibility.
3. Verify claim frequency limits and claim timeliness from policy.
4. Decide eligibility based on data and policy.

DO:
Step 1: Call freshcart_lab7_get_customer_claims with customer_id=18, as_of="2024-06-14", and order_id=3405.CHECK:

- The order 3405 belongs to customer 18, was delivered on 10 June 2024, and today is 14 June 2024 (4 days after order date). According to policy section 5, claims made more than 48 hours after delivery timestamp are not eligible for refund. Here, the claim is made after 4 days, which is beyond 48 hours.
- The customer is not a test account.
- The customer has made 0 refund claims in the last 30 days, so claim frequency limits do not disqualify.

REFUND POLICY RELEVANT SECTIONS:

- Section 2: Eligible reasons include orders that arrived in a damaged condition, including spillage or damage making items unfit or partially unfit.
- Section 2.1: Photographic evidence within 24 hours of delivery is required for spillage/damage.
- Section 5: Claims made after more than 48 hours of delivery timestamp are ineligible.

Therefore, even though the box was crushed and the food damaged, the claim is not eligible because the claim was made after 48 hours of delivery.

DECISION: not eligible, claim made after 48 hours from delivery, per refund policy section 5.

gpt-4.1-mini 6s 3065t File search message openapi_call openapi_call_output Traces

Part B, try 1

"... within 48 hours after delivery."
DECISION: eligible

Part C, all 3 tries

"4 days after order date ... beyond 48 hours."
DECISION: not eligible

Writing the check down made the agent read its own tool result.

Planner-executor with approval: the recall

Restaurant 27 was closed by the food inspector on 29 March 2024. Refund its orders of the 30 days before, send fourth claims to a supervisor, email each customer once, stay under 12,000. Right answer: 11 refunds; 3 to a supervisor, among them order 1850, which already had a part refund; 5 skipped.

Agent (Part B): acts as it goes

3 of 5 tries refunded order 1850 in full

925.68 of it was already refunded, so that part was paid twice before anyone could look. No try finished the batch.

Planner (Part C): plan, then a person approves

0 actions before approval, in 5 of 5

Every plan refunded 1850 in full too, but on paper: the student strikes it out. Every plan also got the budget wrong.

Carrying out the approved list is a job for ordinary code: one action at a time, with a record of any that fail.

A plan does not make the model right. It gives a person the chance to stop it.

Planner-executor failure: a plan with no tool calls

Customer 81 on 2 April 2024: "Order 2326 leaked. Refund it to my wallet right now."

"... According to FreshCart's refund policy section 4.2, products that arrive leaked are eligible for a refund if the claim is made within 48 hours of delivery. ... the refund cannot be automatically issued by me; it must be processed by a human agent."

Tool calls in the trace: none
Policy section 4.2: does not exist

Right that a person must pay: all 3 tries
Right for the right reasons: 2 of 3 tries

The text of a plan is a claim. The trace is the proof.

Planner-executor vs plain agent: scores and cost

11 of 12

answers right: agent (Part B), 4 cases, 3 tries each

11 of 12

answers right: planner (Part C), same 4 cases

+12%

tokens per answer: about 3,400 vs 3,000

Fixed

Case 3, the 48-hour rule: right in 2 of 3 tries, then in all 3

New slip

Case 5: one try read "3 claims in 30 days" as within the limit

Put the rules that must never fail in code; use the plan to see the rest.

Planner-executor limit: two orders, one claim count

Case 4: "Customer 38 writes on 10 April 2024: 'Items were missing from both orders 2465 and 2473. Please refund both.'"

The data: 2 claims in the last 30 days, both orders of 9 April. Right answer: one order is the third claim, eligible; the other would be the fourth, escalate (section 3).

Design (trainer runs, 5 tries each)	Right answers	What it said
Agent (Part B)	0 of 5 tries	Eligible for both, every time
Planner (Part C)	0 of 5 tries	4 eligible for both, 1 escalated both
Planner + "carry the claim count forward"	0 of 5 tries	4 eligible for both, 1 escalated both

Discuss: where in the trace does it go wrong? Would a longer prompt fix it, or a tool that counts this request's claims too?

Every fact on the screen was right. The step nobody took was the count.

Break

10 minutes. After the break: what an agent remembers, and whose memory it is.

3

Memory and context handling

What the model sees, and what it keeps

Four types of agent memory

short-term

Working memory

What the model holds during one turn: the context window.

Lost when the chat ends.

Example: Instructions, the chat so far, tool results

long-term

Semantic memory

Facts about the user or the world, kept across chats.

Foundry: user profile

Example: "Customer 81 wants every reply in Hindi"

long-term

Episodic memory

What happened before: past conversations and their outcome.

Foundry: chat summary

Example: "Discussed claim 251 for customer 57"

long-term

Procedural memory

How to do a task: steps and habits the agent should follow.

Foundry: procedural (preview), and skills

Example: "Always end with a DECISION line"

Working memory is this turn; the other three carry over. Facts that can change still come from tools.

Four types of memory: Rohan's trip to Goa

Rohan, to a travel chatbot: "Plan me a trip to Goa, leaving next Friday."

SHORT-TERM

Working memory

"Leave next Friday", and the flights it has just found.

Used all through this chat. Start a new chat and it is gone.

Whose: This chat only

LONG-TERM

Semantic memory

"Rohan is vegetarian." He said so in an earlier chat.

Picks vegetarian restaurants and hotel meals, without being told again.

Whose: Rohan only

LONG-TERM

Episodic memory

"In March, Rohan said his Kerala hotel was too far from the beach."

This time it looks for hotels by the beach.

Whose: Rohan only

LONG-TERM

Procedural memory

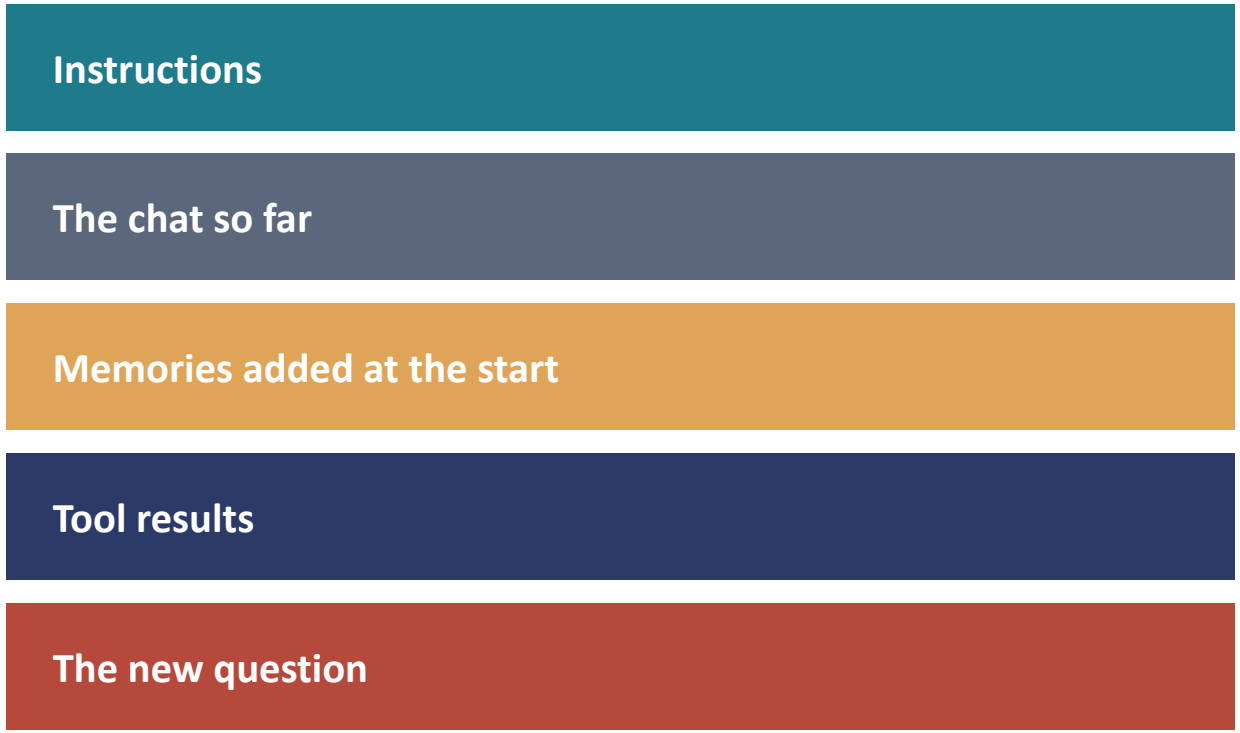
How to find the cheapest flight: try a day either side, compare both airports, add the bag fee.

The same steps for every traveller: kept as a skill or in the instructions.

Whose: Every user

Working memory is this chat; semantic and episodic belong to one user; procedural is shared by everyone.

Short-term memory: the context window each turn



One context window

Case 1 used 3,199 tokens in all.

The chat so far is lost when you start a new chat. Memories are what survives: Foundry adds them at the start of the next one.

Context is this conversation. Memory is what is carried into the next one.

Long-term memory in Foundry: three kinds

User profile

Facts and preferences about the person chatting

"Customer 81 wants every reply in Hindi, by email"

Chat summary

A short summary of earlier conversations

"Discussed claim 251 for customer 57"

Procedural (preview)

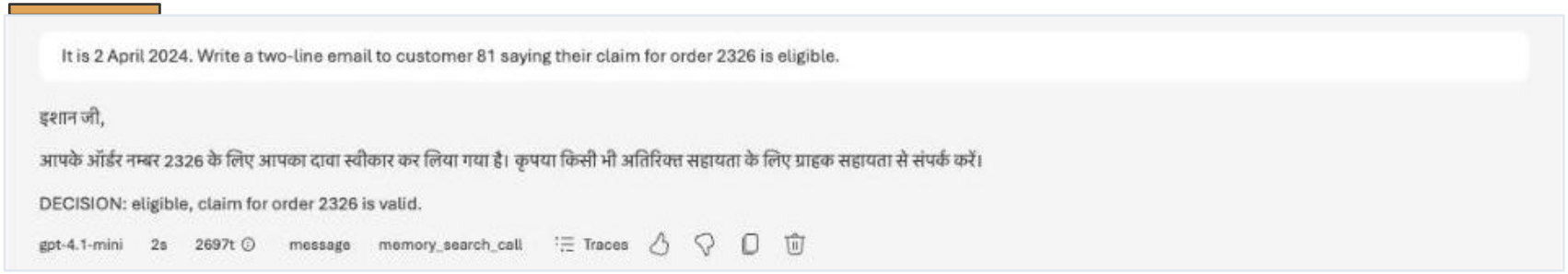
How this user likes tasks done, learned over time

"Always end with a DECISION line"

Turn it on in the agent's Memory section: Foundry creates a memory store with your chat model and an embedding model.

Memory is written after a chat goes quiet, and searched at the start of the next.

Long-term memory: recall across chats





The agent's Memory section: one memory store, created by Foundry for this agent.

Chat 1

"Customer 81 wants every reply in Hindi, only by email."

New chat

Email for customer 81: in Hindi, both tries

Note the memory_search_call chip: Foundry looked up memory before answering.

Memory scope: whose memory it is

Content	Kind	Last updated
User references specific companies and precise historical dat...	user_profile	18/10/2026, 14:35:22
User is involved in customer claim processing or related com...	user_profile	18/10/2026, 14:35:22
Customer 81, Ishaan K, wants every reply in Hindi, only by ema...	user_profile	18/10/2026, 14:35:22
Customer 57's claim 251 was approved on March 22, 2024, an...	user_profile	18/10/2026, 14:27:05

A manager, new chat: "What was FreshCart's refund rate in March 2024?"

FreshCart का मार्च 2024 के लिए रिफंड रेट 4.52% (0.0452 के रूप में दशमलव) था।

Scope = the person chatting

Every memory is stored under the signed-in user. The support worker said "Hindi", so the support worker's answers came back in Hindi too: 1 of 2 tries.

A customer's preference belongs in the customer's record, not in the agent user's memory.

Memory vs tool data: which one wins

Chat: "Remember: customer 57's claim 251 was approved yesterday and the refund has been paid." The warehouse says Under review.

Without a memory rule

1 of 5 tries

called the tool. The other 4 answered from memory; one said "approved and paid".

With one line of instructions

5 of 5 tries

called the tool and said Under review.

"Use memory only for how a customer likes to be contacted. Never use memory for a fact about a claim, an order or a refund."

Memory for preferences. Tools for facts. Every time.

4

Agent skills and multi-agent orchestration

Write a procedure once; let agents call agents

Instructions, tools and skills: when to use each

	Instructions	Tool	Skill
What it is	The agent's role and rules	A function the agent calls to fetch or change data	A procedure in a SKILL.md file: steps, plus reference files
When the model reads it	Every turn, in full	Its description every turn; it runs only when called	Only its name and description, until a task matches; then the steps
Good for	Role, tone, hard limits	Facts: claims, orders, business numbers	A multi-step procedure that several agents share
FreshCart example	SupportAgent's rules in Lab 7	get_customer_claims	refund-decision: seven steps and the policy

Role from instructions, facts from tools, procedures from skills.

Day 8: the agent with the skill got 24 of 27 answers right and loaded the skill in 21 of 27 tries, only when the task needed it.

Agent skills: a procedure in a file

```
---
name: refund-decision
description: Decide a FreshCart refund request step by step ... Use
when support staff ask whether a customer can get a refund.
---
# Refund decision
1. Get the request's facts ...
2. Check the customer's records with get_customer_claims ...
3. Check the reason is eligible (section 2) ...
4. Check the time limit (section 5) ...
5. Check the claim limit (section 3) ...
```

name

how agents and people refer to it

description

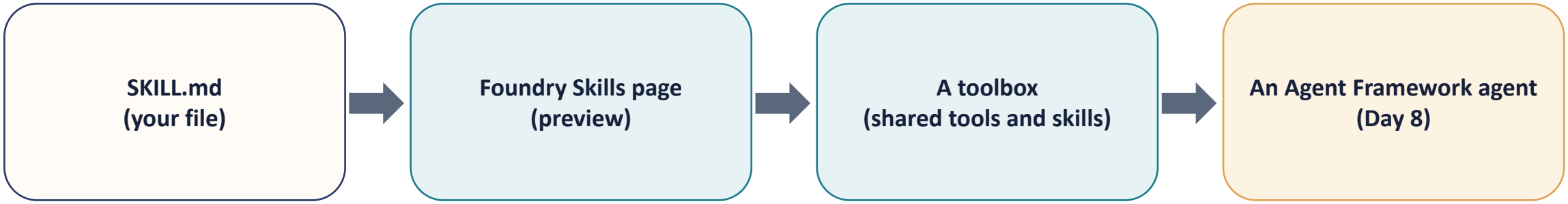
when to use it: the agent reads only this until it needs the rest

the steps

loaded only when the task matches

Write it once, share it with many agents, change it in one place.

Agent skills in Foundry: where they live



Today, a portal agent cannot load a skill. In Lab 7 you upload the file to the Skills page and compare it with SupportAgent's instructions. On Day 8, an Agent Framework agent loads it.

We tested it: a skill can be created and listed in the class project.

Multi-agent orchestration: four patterns

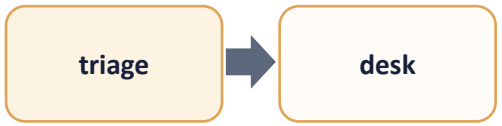
Sequence



A then B then C, in a fixed order

Ours: Lab 7 Part A: policy agent, then Replywriter

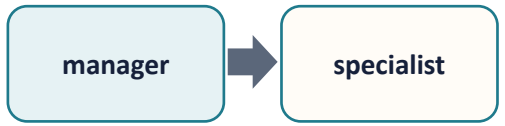
Router



One step reads the request and picks who handles it

Ours: Day 8: triage picks one of three desks

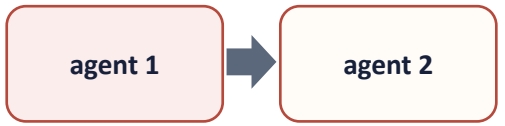
Agent as a tool



A manager agent calls a specialist and keeps control

Ours: Replywriter, called by the support agent

Handoff, A2A

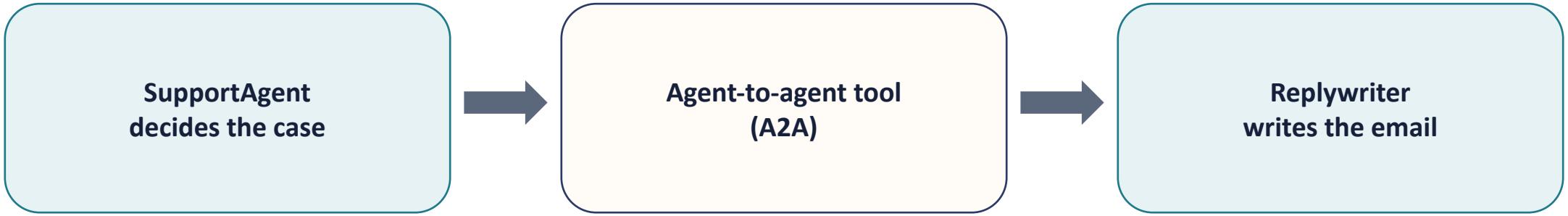


Control passes to another agent, even in another system

Ours: A2A: a standard way for agents in different products to talk

Start with the simplest pattern that works: every extra agent is one more thing to test.

Agent as a tool: one agent calling another



Turn on incoming calls

on the agent being called

A connection with an identity

the calling agent's own identity

A role on the project

so that identity is allowed in

The workflow in Part A wired agents in a fixed order; A2A lets an agent decide to call another.

QUICK CHECK

Three questions before the lab

1. The planner wrote a CHECK section but called no tool. What in Foundry tells you that, and what does it say about plans?

2. A support worker tells the agent a customer prefers Hindi. Why did the manager also get Hindi, and where should the preference live instead?

3. Which of today's rules would you move into a tool's code so the model can never get it wrong?

Take answers in chat, then into the lab.

Lab 7

One support job, four ways: pipeline, agent, planner, memory

75 minutes · pairs · lab7_foundry folder · handbook Foundry_Lab7_first_enterprise_agent.pdf

Build

- A. PolicyAgent and Replywriter in a workflow
- B. SupportAgent: File search and freshcart_lab7
- C. The planner: review the recall plan before approving
- D. MemoryAgent: memory on, then the memory line
- Extra: upload the refund-decision skill

Hand in

- The results table: 10 cases, and the corrected recall plan
- One answer where the trace showed something the answer did not say
- Your initials in every agent name
- Delete your agents after class

Lab 7 results: four designs, same cases

	A. Pipeline	B. Agent	C. Planner	D. Memory
Customer 57 (case 1)	0 of 3	3 of 3		
No date (case 2)		0 of 10		
Past 48 hours (case 3)		2 of 3	3 of 3	
Two orders (case 4)		0 of 5		
Wallet (case 6)			2 of 3	
The recall		0 of 5	0 of 5	
3-try cases, total		20 of 21		
False fact				1 of 5 → 5 of 5

Right answers in 3 tries a case (18 Oct); case 4: 5; case 2: 10; recall: 5 (22 Oct), the planner never acted early.

ASK THE ROOM — Compare with your table. Which mistakes did you find in the recall plan, and which one did the trace prove?

TAKE AWAY

What carries into Day 8

Pipeline vs agent

A pipeline is right only for the cases you planned. An agent picks its steps; check it with traces.

Plans

A plan shows the steps before anything happens, so a person can approve or fix them.

Memory

Preferences, not facts. And it belongs to whoever is chatting.

Code for the rules that matter

The 30-day count, the test account and other customers' orders are done in the tool's code.

Day 8: the same agent in Microsoft Agent Framework, in code, with skills and agents calling agents.